

NUMPY & PANDAS — DATA ANALYTICS ASSIGNMENT

Complete Practical Solution / PDF Report

Objective: Develop practical skills in NumPy and Pandas for array operations, data manipulation, inspection, cleaning, filtering, sorting, merging, GroupBy, Pivot Tables, and exporting processed data.

How to use this solution: The programs below are ready to copy into Jupyter Notebook, Google Colab, or VS Code. Task 5 onward uses a small CSV dataset created in Python so the assignment remains completely reproducible.

Task 1: NumPy Introduction & Arrays (10 Marks)

This program imports NumPy, creates an array of more than 10 numbers, displays its shape, size and data type, and creates both one-dimensional and two-dimensional arrays.

```
import numpy as np

# NumPy array containing 12 numbers
arr = np.array([10, 20, 30, 40, 50, 60, 70, 80, 90, 100, 110, 120])

print("Array:", arr)
print("Shape:", arr.shape)
print("Size:", arr.size)
print("Data type:", arr.dtype)

# One-dimensional array
one_d = np.array([1, 2, 3, 4, 5])
print("\nOne-dimensional array:")
print(one_d)

# Two-dimensional array
two_d = np.array([[1, 2, 3], [4, 5, 6]])
print("\nTwo-dimensional array:")
print(two_d)
print("2-D shape:", two_d.shape)
```

Expected key output: The 12-element array has shape **(12,)**, size **12**, and an integer data type. The two-dimensional array has shape **(2, 3)**.

Task 2: NumPy Indexing, Slicing & Reshaping (10 Marks)

```
import numpy as np

arr = np.array([10, 20, 30, 40, 50, 60, 70, 80, 90, 100])

# Indexing
print("First element:", arr[0])
print("Fifth element:", arr[4])
print("Last element:", arr[-1])

# Slicing
print("Elements from index 2 to 6:", arr[2:7])
print("Every second element:", arr[::2])

# Two-dimensional array
matrix = np.array([
    [1, 2, 3, 4],
    [5, 6, 7, 8],
    [9, 10, 11, 12]
])

print("\n2-D array:")
print(matrix)

# Specific rows and columns
print("First row:", matrix[0])
print("Second column:", matrix[:, 1])
print("Element at row 3, column 2:", matrix[2, 1])

# Reshaping
original = np.arange(1, 13)
reshaped_3x4 = original.reshape(3, 4)
reshaped_2x6 = original.reshape(2, 6)

print("\nOriginal array:")
print(original)
print("\nReshaped to 3 x 4:")
print(reshaped_3x4)
print("\nReshaped to 2 x 6:")
print(reshaped_2x6)
```

Important: reshaping does not change the total number of elements. For example, 12 elements can be reshaped into 3×4 or 2×6.

Task 3: NumPy Mathematical & Statistical Operations (10 Marks)

```
import numpy as np

data = np.array([10, 20, 30, 40, 50])

# Mathematical operations
print("Addition:", data + 5)
print("Subtraction:", data - 5)
print("Multiplication:", data * 2)
print("Division:", data / 2)

# Statistical operations
print("\nMean:", np.mean(data))
print("Median:", np.median(data))
print("Minimum:", np.min(data))
print("Maximum:", np.max(data))
print("Standard Deviation:", np.std(data))
print("Sum:", np.sum(data))
```

For the dataset [10, 20, 30, 40, 50], the mean and median are 30, minimum is 10, maximum is 50, and sum is 150. NumPy performs these calculations efficiently on arrays.

Task 4: Pandas Series & DataFrame (10 Marks)

```
import pandas as pd

# Pandas Series
marks = pd.Series([85, 78, 92, 88, 76],
                  index=["Aman", "Riya", "Karan", "Neha", "Arjun"])

print("Pandas Series:")
print(marks)

# DataFrame
students = pd.DataFrame({
    "Name": ["Aman", "Riya", "Karan", "Neha", "Arjun"],
    "Age": [20, 21, 20, 22, 21],
    "Marks": [85, 78, 92, 88, 76],
    "City": ["Delhi", "Lucknow", "Noida", "Delhi", "Meerut"]
})

print("\nDataFrame:")
print(students)

print("\nColumn names:")
print(students.columns)

print("\nIndex:")
print(students.index)

# Add a new column
students["Result"] = ["Pass", "Pass", "Pass", "Pass", "Pass"]

print("\nUpdated DataFrame:")
print(students)
```

A Series is a one-dimensional labeled data structure, while a DataFrame is a two-dimensional table with rows and columns.

Task 5: Reading & Inspecting Data (10 Marks)

First create a small CSV file named **students.csv**. This dataset is also suitable for Tasks 6–10.

```
import pandas as pd

data = {
    "Student_ID": [101, 102, 103, 104, 105, 106, 107, 108],
    "Name": ["Aman", "Riya", "Karan", "Neha", "Arjun", "Priya", "Rohit", "Simran"],
    "City": ["Delhi", "Lucknow", "Noida", "Delhi", "Meerut", "Noida", "Lucknow", "Delhi"],
    "Department": ["CS", "IT", "CS", "IT", "CS", "IT", "CS", "IT"],
    "Marks": [85, 78, 92, 88, 76, 95, 69, 84],
    "Attendance": [90, 85, 96, 91, 80, 98, 72, 89]
}

df_create = pd.DataFrame(data)
df_create.to_csv("students.csv", index=False)

# Read CSV
df = pd.read_csv("students.csv")

print("First 5 rows:")
print(df.head())

print("\nLast 5 rows:")
print(df.tail())

print("\nNumber of rows and columns:", df.shape)

print("\nColumn names:")
print(df.columns)

print("\nData types:")
print(df.dtypes)

print("\nInformation:")
df.info()

print("\nDescriptive statistics:")
print(df.describe())
```

Useful inspection methods: **head()** previews the beginning, **tail()** previews the end, **shape** gives dimensions, **dtypes** gives data types, **info()** gives structural information, and **describe()** gives numerical summary statistics.

Task 6: Selecting, Filtering & Sorting Data (10 Marks)

```
import pandas as pd

df = pd.read_csv("students.csv")

# Select specific columns
print("Selected columns:")
print(df[["Name", "Marks"]])

# Select specific rows
print("\nRows 0 to 2:")
print(df.iloc[0:3])

# Filter: marks greater than 80
high_marks = df[df["Marks"] > 80]
print("\nStudents with marks > 80:")
print(high_marks)

# Multiple conditions
filtered = df[(df["Marks"] > 80) & (df["Attendance"] > 90)]
print("\nMarks > 80 AND Attendance > 90:")
print(filtered)

# Ascending order
print("\nMarks ascending:")
print(df.sort_values(by="Marks", ascending=True))

# Descending order
print("\nMarks descending:")
print(df.sort_values(by="Marks", ascending=False))
```

Boolean filtering is useful for selecting records that satisfy business or analytical conditions. Multiple conditions can be combined with **&** for AND and **|** for OR.

Task 7: Handling Missing Values (10 Marks)

```
import pandas as pd
import numpy as np

# Dataset containing missing values
missing_df = pd.DataFrame({
    "Name": ["Aman", "Riya", "Karan", "Neha", "Arjun"],
    "Marks": [85, np.nan, 92, 88, np.nan],
    "Attendance": [90, 85, np.nan, 91, 80],
    "City": ["Delhi", "Lucknow", "Noida", np.nan, "Meerut"]
})

print("Before handling missing values:")
print(missing_df)

# Identify missing values
print("\nMissing-value matrix:")
print(missing_df.isnull())

# Count missing values
print("\nMissing values in each column:")
print(missing_df.isna().sum())

# Remove rows containing any missing value
removed_df = missing_df.dropna()
print("\nAfter removing rows with missing values:")
print(removed_df)

# Fill missing values appropriately
filled_df = missing_df.copy()
filled_df["Marks"] = filled_df["Marks"].fillna(filled_df["Marks"].mean())
filled_df["Attendance"] = filled_df["Attendance"].fillna(
    filled_df["Attendance"].mean()
)
filled_df["City"] = filled_df["City"].fillna("Unknown")

print("\nAfter filling missing values:")
print(filled_df)
```

Why is missing-data handling important? Missing values can produce incorrect calculations, reduce the quality of analysis, and cause problems in machine-learning models. Analysts should identify missing data and choose a suitable method such as removal, mean/median filling, or a meaningful category like "Unknown."

Task 8: Merge, Concatenate, GroupBy & Pivot Table (15 Marks)

For this task, create a second DataFrame containing scholarship information. The student ID is the common column used for merging.

```
import pandas as pd

df = pd.read_csv("students.csv")

# Second dataset
scholarship = pd.DataFrame({
    "Student_ID": [101, 102, 103, 104, 105, 106, 107, 108],
    "Scholarship": ["Yes", "No", "Yes", "No", "Yes", "Yes", "No", "No"],
    "Scholarship_Amount": [5000, 0, 6000, 0, 4500, 7000, 0, 0]
})

# Merge using common column
merged = pd.merge(df, scholarship, on="Student_ID", how="inner")
print("Merged DataFrame:")
print(merged)

# Concatenate
part1 = df.iloc[:4]
part2 = df.iloc[4:]
concatenated = pd.concat([part1, part2], ignore_index=True)

print("\nConcatenated DataFrame:")
print(concatenated)

# GroupBy: department-wise analysis
grouped = df.groupby("Department").agg(
    Average_Marks=("Marks", "mean"),
    Maximum_Marks=("Marks", "max"),
    Minimum_Marks=("Marks", "min"),
    Student_Count=("Student_ID", "count")
).reset_index()

print("\nDepartment-wise GroupBy:")
print(grouped)

# Pivot Table
pivot = pd.pivot_table(
    df,
    values="Marks",
    index="Department",
    columns="City",
    aggfunc="mean",
    fill_value=0
)

print("\nPivot Table:")
print(pivot)
```

Merge joins tables using a common key. **Concatenate** stacks DataFrames along rows or columns. **GroupBy** divides records into groups and applies aggregate functions. A **Pivot Table** summarizes values across row and column categories.

Task 9: Exporting Data (5 Marks)

```
import pandas as pd

df = pd.read_csv("students.csv")

# Example processing
df["Performance"] = df["Marks"].apply(
    lambda x: "Excellent" if x >= 90
    else "Good" if x >= 80
    else "Needs Improvement"
)

# Export processed DataFrame
output_file = "processed_students.csv"
df.to_csv(output_file, index=False)

# Verify exported file
verified_df = pd.read_csv(output_file)

print("Exported file:", output_file)
print("\nVerified processed data:")
print(verified_df)
```

The file **processed_students.csv** is created in the current working directory. Open it in Excel or read it again with Pandas to verify that the processed data was saved correctly.

Task 10: Mini Data Analysis Project (10 Marks)

Project Title: Student Performance Analysis using NumPy and Pandas

Dataset: The student dataset used in the previous tasks. The analysis examines marks, attendance, departments, performance levels, and city-wise patterns.

```
import pandas as pd
import numpy as np

# 1. Load dataset
df = pd.read_csv("students.csv")

# 2. Inspect dataset
print("Dataset:")
print(df)
print("\nShape:", df.shape)
print("\nData types:")
print(df.dtypes)
print("\nSummary:")
print(df.describe())

# 3. Check and handle missing values
print("\nMissing values:")
print(df.isna().sum())

# 4. Select and filter
print("\nName and Marks:")
print(df[["Name", "Marks"]])

high_performers = df[df["Marks"] >= 85]
print("\nStudents scoring 85 or above:")
print(high_performers)

# 5. Sort
sorted_df = df.sort_values("Marks", ascending=False)
print("\nStudents sorted by marks:")
print(sorted_df)

# 6. NumPy analysis
marks = df["Marks"].to_numpy()
print("\nMean marks:", np.mean(marks))
print("Median marks:", np.median(marks))
print("Highest marks:", np.max(marks))
print("Lowest marks:", np.min(marks))
print("Standard deviation:", np.std(marks))

# 7. GroupBy analysis
dept_analysis = df.groupby("Department").agg(
    Average_Marks=("Marks", "mean"),
    Average_Attendance=("Attendance", "mean"),
    Student_Count=("Student_ID", "count")
).reset_index()

print("\nDepartment-wise analysis:")
print(dept_analysis)

# 8. Pivot Table
pivot = pd.pivot_table(
    df,
    values="Marks",
    index="Department",
    columns="City",
    aggfunc="mean",
    fill_value=0
)

print("\nPivot Table:")
print(pivot)

# 9. Add performance category
df["Performance"] = pd.cut(
    df["Marks"],
    bins=[0, 69, 79, 89, 100],
    labels=["Needs Improvement", "Average", "Good", "Excellent"]
)
```

```
print("\nPerformance categories:")
print(df[["Name", "Marks", "Performance"]])

# 10. Export cleaned/processed data
df.to_csv("final_processed_students.csv", index=False)
print("\nFinal dataset exported as final_processed_students.csv")
```

Findings / Insights

1. The average marks of the complete dataset can be calculated using NumPy's mean function.
2. The student with the highest marks can be identified by sorting the Marks column in descending order.
3. GroupBy shows whether the CS or IT department has a higher average score and attendance.
4. The Pivot Table shows average marks for each department across cities.
5. Attendance can be compared with marks to identify students who perform strongly while maintaining high attendance.
6. The processed dataset contains a Performance category and is exported for further analysis.

Complete Combined Program (Optional)

The following compact script can be used after creating students.csv. It demonstrates the main Pandas/NumPy workflow in one place.

```
import numpy as np
import pandas as pd

# Load
df = pd.read_csv("students.csv")

# Inspect
print(df.head())
print(df.shape)
print(df.dtypes)
print(df.isna().sum())

# Clean
df["Marks"] = df["Marks"].fillna(df["Marks"].median())
df["Attendance"] = df["Attendance"].fillna(df["Attendance"].median())

# Select and filter
print(df[["Name", "Marks"]])
print(df[(df["Marks"] >= 80) & (df["Attendance"] >= 85)])

# Sort
df = df.sort_values("Marks", ascending=False)

# NumPy statistics
marks = df["Marks"].to_numpy()
print("Mean:", np.mean(marks))
print("Median:", np.median(marks))
print("Min:", np.min(marks))
print("Max:", np.max(marks))
print("Std:", np.std(marks))
print("Sum:", np.sum(marks))

# GroupBy
print(df.groupby("Department")["Marks"].agg(["count", "mean", "min", "max"]))

# Pivot Table
print(pd.pivot_table(
    df, values="Marks", index="Department",
    columns="City", aggfunc="mean", fill_value=0
))

# Export
df.to_csv("final_processed_students.csv", index=False)
print("Saved: final_processed_students.csv")
```

Submission Checklist

- Task 1 NumPy arrays completed
- Task 2 Indexing, slicing and reshaping completed
- Task 3 Mathematical and statistical operations completed
- Task 4 Series and DataFrame completed
- Task 5 CSV reading and inspection completed
- Task 6 Selection, filtering and sorting completed
- Task 7 Missing values identified, removed and filled
- Task 8 Merge, concatenate, GroupBy and Pivot Table completed
- Task 9 Processed CSV exported and verified
- Task 10 Mini analysis project and insights completed
- Submit this PDF along with the generated CSV files if required by your instructor

Required Python packages: **numpy** and **pandas**. Install with: **pip install numpy pandas**